

Alberi

Riccardo Di Michele

18 settembre 2026

Introduzione

Usiamo questa *struttura dati* per conservare i nostri dati in maniera **gerarchica**, seguendo un certo ordine. Un **albero** è composto da **nodi**, ciò che contiene i nostri dati e da **archi**, ciò che collega i nostri *nodi*. I nodi possono essere chiamati:

- **padre**: quando da quel nodo *parte* un arco verso un'altro, o altri nodi
- **figlio**: quando in quel nodo *entra un solo* arco
- **radice**: ovvero l'unico nodo dell'albero **senza un padre**
- **fratelli**: nodi figlio che condividono lo stesso padre
- **foglie**: nodi posti alla "fine" della struttura che non presentano figli

Infine useremo il termine **altezza** dell'albero per intendere la distanza massima tra la *radice* e la *foglia* più profonda, e il termine **profondità** per intendere il numero di archi (o *passaggi*) da attraversare per arrivare dalla radice a un determinato nodo v .

Tipologie di alberi

Rappresentazioni indicizzate per alberi

Tutte le seguenti rappresentazioni hanno un **costo di spazio** pari a $O(n)$, dove n è ovviamente la grandezza dell'array usato. Sono strutture indicate per alberi statici poichè l'**inserimento** e **cancellazione** dei nodi (tranne rari casi) può raggiungere costi elevati.

Un loro grande vantaggio è però l'accesso alle info

Vettore dei padri

Questa rappresentazione è molto generica e usata principalmente in alberi statici, la cui forma non cambia. Si basa sul conservare l'indice relativo al padre di ogni nodo all'interno del **nodo stesso**.

Ogni elemento del **vettore padri** sarà quindi un nodo contenente la coppia *info-parent*. Questo approccio porta ovviamente un'ipotetica funzione $trovaPadre(v)$ ad avere un costo computazionale pari a $O(1)$, ma ad avere invece, una funzione $trovaFiglio(v)$ con costo $O(n)$, nel caso il figlio fosse alla "fine" della struttura.

Altre operazioni potrebbero però risultare complesse, come l'**inserimento di un nodo**: possiamo facilmente *appendere* il nuovo nodo all'array (o in caso *riallocare spazio*) ma dobbiamo comunque rispettare la struttura dell'*array padri* cercando il padre del nuovo nodo inserito, portando quindi l'operazione ad un costo nel caso peggiore di $O(n)$.

Nel caso della **eliminazione di un nodo** le cose si complicano. Avremo sempre un costo $O(n)$, poichè dobbiamo potenzialmente scorrere tutto l'array per trovare il nodo da cancellare, ma non solo una volta cancellato, in base alla strategia scelta, avremo costi in più.

Indice	Info	Parent
0	A	null
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	1
7	H	2
8	I	2
9	L	2
10	M	3
11	N	3
12	O	3

Vettore posizionale

Questa tipologia è perfetta per gli alberi **d-ari**, ovvero quelle strutture i cui figli di un nodo sono sempre una quantità d , eccetto per i nodi dell'ultimo livello. Preso un albero $T(n, d)$ d-ario, con nodi numerati da 0 a $n - 1$, un **vettore posizionale** P è un array di dimensioni n dove l'elemento $P[v]$ conserva gli elementi del corrispondente nodo v .

L'ordine in cui questi elementi sono conservati all'interno del vettore è molto importante poichè grazie al loro ordine possiamo facilmente trovare qualsiasi figlio del nodo v_i tramite questa formula:

$$P[d \cdot v + i]$$

dove, ricordiamo, d essere il numero dei figli di ogni nodo, v il nodo di cui vogliamo trovare i figli, e i sarà un valore compreso tra 1 e d .

Per esempio in un albero ternario ($d = 3$) se volessi trovare i figli del nodo $v = 4$ allora avrei:

- $P[3 \cdot 4 + 1] \rightarrow P[13]$, come indice del primo figlio
- $P[3 \cdot 4 + 2] \rightarrow P[14]$, come indice del secondo figlio
- $P[3 \cdot 4 + 3] \rightarrow P[15]$, come indice del terzo figlio

Come nel caso del *vettore padri* anche qui l'**inserimento di un nuovo nodo** è un caso complicato.

Se dovessimo inserire l'ultimo nodo allora possiamo tranquillamente *appendere* il nodo alla fine dell'array, andando a rispettare la struttura posizionale, ma nel caso di un inserimento arbitrario - per esempio "in mezzo" all'array - le cose si fanno molto complicate, portando per esempio a *shiftare* tutti gli altri nodi e, possibilmente, ad avere un costo di $O(1)$.

Lo stesso discorso possiamo farlo con l'istruzione *delete*(v): $O(1)$ se consideriamo l'ultimo nodo o ancora più costoso in base alle strategie usate.

Rappresenzazioni collegate per alberi

Questa tipologia si basa invece sull'idea di conservare i nodi come **record**, ovvero una struttura capace di contenere *dati eterogenei*. Il nodo infatti conterrà in primis l'informazione conservata e dei **puntatori** ad altri nodi in base alla tipologia di rappresentazione scelta.

Danno una maggiore libertà rispetto a quelli *indicizzati* ma presentano delle difficoltà quando dobbiamo **accedere** ai nodi; in questa tipologia caso lo spazio occupato dipende dal tipo di rappresentazione scelto.

Puntatori ai figli

Qui ogni *record* presenta un puntatore per ciascuno dei suoi d -figli, nel caso non avesse un figlio il campo sarà *null*.

Il suo spazio sarà quindi di $O(n \cdot d)$, dove n sono i nodi dell'albero e d i figli di ciascun nodo, che, se costanti, possono semplificare il costo in $O(n)$.

Liste di puntatori ai figli

Ogni nodo questa volta conserva un puntatore ad una **lista** contenente puntatori ai suoi figli. Questa lista è spesso implementata tramite una **struttura collegata**, o, se è noto il *massimo numero di figli*, può essere facilmente implementata tramite un *array*.

Lo spazio occupato è invece pari a $O(1)$: ogni nodo ha una lista che copre una parte (o nessuna) degli n nodi totali, messe insieme però le liste coprono tutti gli n nodi.

Primo figlio-fratello successivo

Infine in questo tipo di rappresentazione ogni nodo conserva un puntatore al suo primo **figlio**, se presente, ed uno al suo prossimo **fratello**, se presente.

Anche qui lo spazio è di $O(n)$.

Considerazioni sui costi computazionali

Nome	Tipo / Uso preferito	Memoria	Accesso Padre	Accesso ai Figli	Insert()	Delete()
Vettore dei padri	Indicizzata / Alberi statici	$O(n)$	$O(1)$	$O(n)$	$O(n)$ se devo trovare indice del padre	$O(n)$ + strategia figli?
Vettore Posizionale	Indicizzata / Alberi statici	$O(n)$	$O(1)$	$O(1)$	$O(1)$ se vincolo su ultimo nodo	$O(1)$ se vincolo ultimo nodo
Puntatori ai Figli	Collegata / Alberi dinamici	$O(n)$	$O(n)$	$O(1)$	$O(n)$ se devo trovare padre	$O(n)$ se devo trovare nodo + strategia figli?
Liste di Puntatori ai figli	Collegata / Alberi dinamici	$O(n)$	$O(n)$	$O(d)$	$O(n)$ se devo trovare padre	$O(n)$ se devo trovare nodo + strategia figli?
Primo figlio-fratello successivo	Collegata / Alberi dinamici	$O(n)$	$O(n)$	$O(d)$	$O(n)$ se devo trovare padre	$O(n)$ se devo trovare padre + strategia figli?

Tipologie di albero

Albero di ricerca binario e AVL

Un albero binario (ogni padre ha al massimo due figli), viene considerato **di ricerca** se preso un qualsiasi nodo v : ogni chiave del suo **sotto-albero sinistro** è minore della chiave di v e se la stessa condizione si applica, al contrario, al **sotto-albero destro**.

Poichè i nodi sono disposti in questo ordine le operazioni come *search()* diventano più semplici avendo un costo nel caso peggiore pari all'altezza dell'albero, ma questo non vuol dire che il costo sia semplice.

Questa tipologia infatti non introduce nessun *bilanciamento*, quindi potremmo avere un caso in cui tutti i nodi dell'albero siano a catena nel sotto-albero sinistro, portando così ad avere un *search* con costo pari a $O(h) = O(n)$.

Proprio per evitare che l'altezza degeneri i nostri costi è stato introdotto l'albero **AVL** (**A**delson-**V**elsky and **L**andis). Questa nuova tipologia introduce un **fattore di bilanciamento**, per ogni nodo infatti viene calcolata la **differenza** tra il suo sotto-albero sinistro e quello destro.

$$\beta = H_L - H_R$$

Per considerarsi *bilanciato* ogni nodo v dovrà avere un fattore di bilanciamento compreso tra 0 e $|1|$.

Heap